

AI Agent 开发圣经

从 LLM 到 Agent (2026)

一本面向开发者的 AI Agent 工程实践书

Generated from the AI-Agent-Book MkDocs source.

目录

首页

00 Preface

- 为什么需要这本书
- 读者对象与学习路径
- AI Agent 工程思维
- 如何使用本书

01 AI Basics

- AI、ML、DL 与 LLM
- Token 与 Embedding
- 训练、推理与上下文
- 能力边界与幻觉

02 Transformer

- Transformer 架构概览
- Attention 与 Self-Attention
- 生成过程与推理参数
- 上下文设计原则

03 Prompt

- Prompt 的组成
- 结构化输出
- 示例与评估
- PromptOps 与版本管理

04 RAG

- RAG 架构
- 切分与元数据
- 检索与重排序
- RAG 评估

05 Agent

- Agent 的定义
- 工具调用
- 记忆与状态
- 规划与执行循环

06 LangGraph

- 为什么使用 LangGraph
- State、Node 与 Edge
- 条件路由与循环控制
- 生产级工作流模式

07 MCP

- MCP 概览
- Host、Client 与 Server
- Tools、Resources 与 Prompts
- MCP 安全边界

08 Projects

- 项目环境与代码组织
- LLM 应用最小闭环

RAG 知识库助手

工具型 Agent 工作流

09 System Design

Agent 系统分层架构

可靠性与容错

成本与延迟优化

观测与评估体系

10 Interview

LLM 基础题

RAG 面试题

Agent 面试题

系统设计题

AI Agent 开发圣经：从 LLM 到 Agent (2026)

这是一本面向开发者的 AI Agent 工程实践书。它从 LLM 基础开始，逐步进入 Prompt、RAG、Agent、LangGraph、MCP、项目实战、系统设计和面试准备。

Reading Path

1. 先理解 LLM 的输入、输出、上下文和能力边界。2. 再学习 Prompt 与 RAG，把模型能力接入可控上下文和外部知识。3. 然后进入 Agent，理解工具调用、记忆、状态、规划和执行循环。4. 最后通过 LangGraph、MCP、项目实战和系统设计，把能力组合成生产级系统。

Chapters

- 00 Preface
- 01 AI Basics
- 02 Transformer
- 03 Prompt
- 04 RAG
- 05 Agent
- 06 LangGraph
- 07 MCP
- 08 Projects
- 09 System Design
- 10 Interview

00 Preface

本章说明本书的定位、读者对象、学习路径与工程实践方式，帮助读者建立从 LLM 到 Agent 的整体学习地图。

Subtopics

- 为什么需要这本书
- 读者对象与学习路径
- AI Agent 工程思维
- 如何使用本书

Navigation

- Next: 01 AI Basics

为什么需要这本书

本节属于《前言》。

Core Idea

AI Agent 正在从演示型应用走向真实业务系统。开发者需要的不只是提示词技巧，而是一套能把模型、工具、数据、状态、评估和部署组织起来的方法。

本节解释本书的写作目标：用工程视角拆解 AI Agent，帮助读者从能调模型，走向能设计、实现、评估和维护 Agent 系统。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

读者对象与学习路径

本节属于《前言》。

Core Idea

本书面向已经具备基础编程能力、希望进入 AI 应用开发和 Agent 工程的读者。你不需要先成为算法研究员，但需要理解模型能力、系统边界和工程约束。

学习路线按“理解模型、组织上下文、接入知识、编排行动、设计系统、准备面试”推进。每一部分都服务于最终目标：构建可靠的 AI Agent。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

AI Agent 工程思维

本节属于《前言》。

Core Idea

Agent 系统不是一次模型调用，而是一组会感知、决策、行动和反馈的流程。工程思维要求我们把不确定的模型能力放进可观测、可回滚、可评估的系统边界里。

本节建立本书反复使用的判断标准：先定义任务，再设计上下文；先约束行动，再扩大自主性；先能评估，再谈优化。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

如何使用本书

本节属于《前言》。

Core Idea

建议读者先通读基础章节，再选择一个项目反复迭代。只阅读概念会低估工程细节，只写代码又容易忽略系统边界。

每章都可以按三个层次使用：先理解概念，再完成小型练习，最后把能力放进完整项目。后续正文会逐步加入代码、图示和案例。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

01 AI Basics

本章建立 AI、机器学习、深度学习与大语言模型的基础概念，为后续理解 Transformer、Prompt、RAG 和 Agent 系统打底。

Subtopics

- AI、ML、DL 与 LLM
- Token 与 Embedding
- 训练、推理与上下文
- 能力边界与幻觉

Navigation

- Next: 02 Transformer

AI、ML、DL 与 LLM

本节属于《AI 基础》。

Core Idea

AI 是目标，机器学习是方法，深度学习是现代机器学习的重要路线，大语言模型则是深度学习在序列建模和语言任务上的代表性成果。

对应用开发者来说，重要的不是背诵定义，而是理解这些层次如何影响系统设计：模型不是数据库，生成不是检索，概率输出不是确定逻辑。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

Token 与 Embedding

本节属于《AI 基础》。

Core Idea

LLM 处理的基本单位通常是 token。文本会先被切分成 token，再映射为向量表示，模型通过这些向量学习语义、语法和上下文关系。

Embedding 让文本可以被比较、聚类 and 检索。RAG 系统中的向量检索，本质上就是利用 embedding 把自然语言问题映射到可计算的语义空间。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

训练、推理与上下文

本节属于《AI 基础》。

Core Idea

训练决定模型的基础能力，推理是在给定输入和参数下生成输出。应用开发主要发生在推理阶段，通过上下文、工具和外部数据改变模型可见的信息。

上下文窗口不是无限记忆。它更像一次任务的工作台：放进去的内容越多，成本越高，噪声也越可能增加。好的系统会筛选上下文，而不是无差别塞入所有信息。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

能力边界与幻觉

本节属于《AI 基础》。

Core Idea

LLM 擅长模式补全、语言生成、归纳总结和代码辅助，但它并不天然知道事实是否最新，也不会自动遵守现实世界的业务约束。

幻觉不是偶发小问题，而是概率生成系统的基本风险。工程上要通过检索、工具调用、结构化输出、验证器和人工审核来降低风险。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

02 Transformer

本章梳理 Transformer

的核心结构与大语言模型的工作原理，重点面向工程开发者理解模型行为与能力边界。

Subtopics

- Transformer 架构概览
- Attention 与 Self-Attention
- 生成过程与推理参数
- 上下文设计原则

Navigation

- Next: 03 Prompt

Transformer 架构概览

本节属于《Transformer》。

Core Idea

Transformer

的关键变化是用注意力机制替代传统循环结构，使模型可以并行处理序列并学习长距离依赖。

现代大语言模型多采用 decoder-only 架构，通过不断预测下一个 token 来生成文本。理解这一点，有助于解释为什么提示词顺序、上下文长度和采样参数会影响输出。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

Attention 与 Self-Attention

本节属于《Transformer》。

Core Idea

Attention 可以理解为模型在生成当前位置表示时，对上下文中不同 token 分配不同权重。Self-Attention 让序列内部的 token 彼此建立关系。

工程上不需要手写注意力公式，但需要知道：模型并不是平均阅读上下文。重要信息的位置、表达方式和重复程度，都会影响它被使用的概率。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

生成过程与推理参数

本节属于《Transformer》。

Core Idea

LLM 输出是一步步生成的。每一步模型都会给出候选 token 的概率分布，再由采样策略决定最终选择。

temperature、top_p、max_tokens 等参数控制输出的随机性、覆盖范围和长度。生产系统通常需要更低随机性、更明确格式和更严格的停止条件。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

上下文设计原则

本节属于《Transformer》。

Core Idea

上下文设计是把任务、约束、资料、示例和输出格式组织给模型的过程。它直接决定模型能看到什么、优先相信什么、如何完成任务。

好的上下文应该短而清晰、结构稳定、来源明确。对于长任务，要把上下文拆成状态、记忆、检索结果和当前指令，而不是依赖一个越来越长的提示词。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

03 Prompt

本章介绍 Prompt Engineering

的结构化方法，帮助开发者把自然语言指令转化为稳定、可复用、可评估的模型输入。

Subtopics

- Prompt 的组成
- 结构化输出
- 示例与评估
- PromptOps 与版本管理

Navigation

- Next: 04 RAG

Prompt 的组成

本节属于《Prompt 工程》。

Core Idea

一个可维护的 Prompt 通常包含角色、目标、上下文、约束、输入数据、输出格式和质量标准。缺少其中任何一项，模型都更容易自由发挥。

工程化 Prompt

不是一句灵感文案，而是系统接口的一部分。它应该能版本化、能测试、能被多人理解和复用。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

结构化输出

本节属于《Prompt 工程》。

Core Idea

生产系统常常需要模型返回 JSON、表格、标签或固定 schema。结构化输出能减少后处理成本，也能让验证器判断结果是否可用。

设计结构化输出时，要明确字段含义、类型、枚举范围和缺省策略。不要只告诉模型“返回 JSON”，还要说明失败时如何表达不确定性。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

示例与评估

本节属于《Prompt 工程》。

Core Idea

Few-shot

示例能把抽象要求变成具体行为边界。好的示例应该覆盖正常情况、边界情况和容易误判的情况。

Prompt 迭代必须配合评估集。没有评估集时，优化容易变成凭感觉调参；有评估集时，每次改动都能看出质量、成本和稳定性的变化。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

PromptOps 与版本管理

本节属于《Prompt 工程》。

Core Idea

Prompt 应该像代码一样管理：有命名、有版本、有变更记录、有回滚方式。尤其在多人协作和生产环境中，隐式改 Prompt 会造成难以定位的行为变化。

PromptOps 的核心不是工具，而是流程：谁能改、改了什么、影响哪些场景、如何验证、如何发布。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

04 RAG

本章讲解检索增强生成的核心架构，覆盖知识库构建、向量检索、重排序与答案生成的工程流程。

Subtopics

- RAG 架构
- 切分与元数据
- 检索与重排序
- RAG 评估

Navigation

- Next: 05 Agent

RAG 架构

本节属于《RAG》。

Core Idea

RAG 把外部知识检索和模型生成结合起来。用户问题先被检索系统转化为候选资料，再由模型基于这些资料生成答案。

RAG 的价值是把事实来源从模型参数转移到可更新、可追踪的知识库。它不能消灭幻觉，但能显著提升可验证性和时效性。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

切分与元数据

本节属于《RAG》。

Core Idea

文档切分决定检索颗粒度。切得太大，会带来噪声；切得太小，会丢失上下文。好的切分通常结合标题、段落、语义边界和业务结构。

元数据是 RAG 的第二索引。时间、来源、权限、产品线、文档类型等字段能帮助系统过滤候选内容，避免把错误资料交给模型。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

检索与重排序

本节属于《RAG》。

Core Idea

向量检索擅长语义相似，关键词检索擅长精确匹配。复杂系统常使用混合检索，再通过重排序模型或规则挑出更可靠的上下文。

检索质量比生成技巧更基础。如果召回内容不对，模型再强也只能在错误材料上生成漂亮但无用的答案。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

RAG 评估

本节属于《RAG》。

Core Idea

RAG 评估至少要看四件事：是否召回正确资料、资料是否被正确引用、回答是否忠于资料、回答是否满足用户任务。

建议把评估分成检索评估和生成评估。前者关注 recall、precision 和 rerank 质量，后者关注 groundedness、完整性和可执行性。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

05 Agent

本章进入 Agent 的核心概念，解释工具调用、记忆、规划、执行循环与多 Agent 协作如何组成可工作的智能系统。

Subtopics

- Agent 的定义
- 工具调用
- 记忆与状态
- 规划与执行循环

Navigation

- Next: 06 LangGraph

Agent 的定义

本节属于《Agent》。

Core Idea

Agent 是能围绕目标感知上下文、选择行动、调用工具并根据反馈继续推进任务的系统。它不等于一个聊天机器人，也不等于一次模型调用。

判断一个系统是否是 Agent，要看它是否有行动空间、状态变化和反馈循环。自主性越高，系统越需要边界、审计和恢复机制。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

工具调用

本节属于《Agent》。

Core Idea

工具让模型从“会说”变成“能做”。搜索、数据库、文件系统、代码执行、工单系统和业务 API 都可以成为工具。

工具设计的关键是收窄权限和明确 schema。模型负责决定何时调用，工具负责执行确定性动作，系统负责校验参数和结果。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

记忆与状态

本节属于《Agent》。

Core Idea

Agent 的记忆不应该只是把历史对话无限追加。更合理的做法是区分短期上下文、长期记忆、任务状态和外部记录。

状态管理决定 Agent 能否恢复、重试和协作。生产级 Agent 应该把关键状态显式保存，而不是把所有过程隐藏在模型上下文里。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

规划与执行循环

本节属于《Agent》。

Core Idea

典型 Agent 循环包括理解目标、制定计划、执行步骤、观察结果、调整计划和输出结论。每个环节都可能失败，因此需要中间检查点。

不要盲目追求全自动。高风险任务应加入人类确认，长任务应支持暂停恢复，外部动作应记录原因、参数和结果。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

06 LangGraph

本章聚焦 LangGraph 的图式编排能力，用状态图、节点、边与检查点构建可控、可恢复的 Agent 工作流。

Subtopics

- 为什么使用 LangGraph
- State、Node 与 Edge
- 条件路由与循环控制
- 生产级工作流模式

Navigation

- Next: 07 MCP

为什么使用 LangGraph

本节属于《LangGraph》。

Core Idea

当 Agent 任务变成长流程、多分支、可恢复和可观测时，简单 while 循环会越来越难维护。LangGraph 用图结构表达工作流，让控制流更加明确。

LangGraph 的价值在于把 Agent 行为拆成节点、边和状态，使开发者可以控制路径、保存进度、处理人工介入和调试执行过程。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

State、Node 与 Edge

本节属于《LangGraph》。

Core Idea

State 是图中共享的数据结构，Node 是处理状态的函数，Edge 决定下一步流向。三者共同定义 Agent 工作流的执行方式。

设计图时应先画状态，再写节点。状态字段越清晰，节点边界越明确，系统越容易测试和演进。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

条件路由与循环控制

本节属于《LangGraph》。

Core Idea

Agent

经常需要根据模型判断、工具结果或业务规则选择下一步。条件路由能把这种选择显式放进图结构中。

循环控制要格外谨慎。必须设置最大步数、失败出口和人工接管路径，避免 Agent 在错误状态中无限尝试。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

生产级 workflows 模式

本节属于《LangGraph》。

Core Idea

生产级 LangGraph 工作流通常需要检查点、幂等工具、异常恢复、人工审核和执行日志。

把图看成业务流程，而不是模型实验脚本。每条边都代表一次系统承诺：为什么进入这里、如何退出、失败时怎么办。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

07 MCP

本章介绍 Model Context Protocol 的设计目标与使用方式，说明 Agent 如何通过标准协议连接外部工具、数据与服务。

Subtopics

- MCP 概览
- Host、Client 与 Server
- Tools、Resources 与 Prompts
- MCP 安全边界

Navigation

- Next: 08 Projects

MCP 概览

本节属于《MCP》。

Core Idea

MCP 是一种开放标准，用来让 AI 应用以一致方式连接外部系统，例如文件、数据库、搜索、业务工具和工作流。

可以把 MCP 理解为 Agent 与外部能力之间的标准接口层。它不替代业务 API，而是让不同 AI 客户端更容易发现和调用这些能力。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

Host、Client 与 Server

本节属于《MCP》。

Core Idea

在 MCP 架构中，用户面对的是 Host，Host 内部通过 Client 与一个或多个 MCP Server 通信。Server 暴露工具、资源和提示模板。

这种分层让能力提供方和 AI 应用方解耦。开发者可以独立开发 MCP Server，再被不同支持 MCP 的客户端复用。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

Tools、Resources 与 Prompts

本节属于《MCP》。

Core Idea

Tools 用于执行动作，Resources 用于暴露可读取的上下文，Prompts 用于提供可复用的任务模板。三者对应 Agent 系统中的行动、知识和指令。

设计 MCP Server

时，要清楚每个能力属于哪一类。能读取的不一定能执行，能执行的必须有更严格的权限和审计。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

MCP 安全边界

本节属于《MCP》。

Core Idea

MCP 把工具接入变得更方便，也把安全边界变得更重要。任何能访问文件、命令、网络或业务系统的 Server 都需要权限控制。

生产环境应考虑最小权限、用户确认、输入校验、输出过滤、日志审计和供应链治理。Agent 能调用工具，不代表它应该无条件调用工具。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

08 Projects

本章规划一组从入门到进阶的实战项目，把 Prompt、RAG、Agent、LangGraph 与 MCP 串成可运行的工程案例。

Subtopics

- 项目环境与代码组织
- LLM 应用最小闭环
- RAG 知识库助手
- 工具型 Agent workflow

Navigation

- Next: 09 System Design

项目环境与代码组织

本节属于《项目实战》。

Core Idea

AI Agent 项目应该从一开始就区分应用层、模型层、工具层、数据层和评估层。目录结构清晰，后续扩展才不会变成脚本堆叠。

建议所有示例项目都包含配置文件、README、测试样例、评估数据和运行脚本。即使是教学项目，也应该模拟真实工程约束。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

LLM 应用最小闭环

本节属于《项目实战》。

Core Idea

第一个项目目标是完成一个可运行的 LLM 应用：接收输入、构造 Prompt、调用模型、解析输出、记录日志。

这个项目看似简单，却覆盖 AI 应用的基本骨架。后续 RAG 和 Agent 都是在这个闭环上增加检索、工具、状态和控制流。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

RAG 知识库助手

本节属于《项目实战》。

Core Idea

第二个项目构建一个知识库问答助手。它需要文档导入、切分、向量化、检索、上下文拼装和基于资料的回答。

项目重点不是界面，而是可验证性。回答应该能追踪到引用资料，无法回答时应明确说明缺少依据。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

工具型 Agent workflow

本节属于《项目实战》。

Core Idea

第三个项目让 Agent 调用工具完成多步骤任务，例如搜索资料、整理结果、生成报告或检查代码。

项目重点是控制工具权限和记录执行轨迹。每次工具调用都应知道输入、输出、失败原因和下一步决策。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

09 System Design

本章从生产系统角度讲解 AI Agent 的架构设计，覆盖可靠性、成本、延迟、安全、观测与评估体系。

Subtopics

- Agent 系统分层架构
- 可靠性与容错
- 成本与延迟优化
- 观测与评估体系

Navigation

- Next: 10 Interview

Agent 系统分层架构

本节属于《系统设计》。

Core Idea

生产级 Agent 系统通常包括入口层、编排层、模型层、工具层、数据层、评估层和观测层。分层不是为了复杂，而是为了隔离变化。

模型会换，工具会增，业务规则会变。清晰分层能让系统在变化时只修改局部，而不是牵一发而动全身。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

可靠性与容错

本节属于《系统设计》。

Core Idea

Agent

系统的失败模式很多：模型误判、工具超时、数据缺失、权限不足、输出格式错误、循环无法停止。

可靠性设计要从失败开始。每个关键步骤都应有超时、重试、降级、人工接管和可恢复状态。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

成本与延迟优化

本节属于《系统设计》。

Core Idea

AI Agent 往往比普通 LLM

调用更贵，因为它可能多轮推理、多次检索、多次工具调用。成本和延迟必须作为系统指标设计。

常见优化包括模型分层、缓存、批处理、上下文压缩、提前终止、异步执行和只在必要时调用昂贵工具。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

观测与评估体系

本节属于《系统设计》。

Core Idea

没有观测就无法调试 Agent，没有评估就无法证明改动变好。生产系统需要记录输入、上下文、模型输出、工具调用、状态变化和最终结果。

评估应同时覆盖离线样例集和线上真实任务。指标不仅包括回答质量，还包括成功率、人工接管率、成本、延迟和安全事件。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

10 Interview

本章整理 AI Agent

开发相关的面试知识框架，帮助读者把概念、项目经验与系统设计能力组织成可表达的答案。

Subtopics

- LLM 基础题
- RAG 面试题
- Agent 面试题
- 系统设计题

Navigation

- [Back to Home](#)

LLM 基础题

本节属于《面试准备》。

Core Idea

面试中的 LLM 基础题通常考察 token、上下文、推理参数、幻觉、结构化输出和模型评估。回答时要避免只背概念，要结合工程影响。

例如解释 temperature

时，不只说它控制随机性，还要说明为什么客服、代码生成和创意写作会使用不同设置。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

RAG 面试题

本节属于《面试准备》。

Core Idea

RAG 面试重点在知识库构建、切分策略、检索质量、重排序、引用、权限和评估。优秀答案应能说明每个环节的失败模式。

当被问到“如何优化 RAG 效果”时，应先定位问题在召回、排序、上下文还是生成，而不是直接说换更大的模型。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

Agent 面试题

本节属于《面试准备》。

Core Idea

Agent 面试会关注工具调用、记忆、规划、多步骤执行、失败恢复和安全边界。回答时要强调 Agent 是系统，不只是模型。

一个成熟答案应该包含：任务拆解、工具 schema、状态管理、执行日志、人工确认、评估指标和异常处理。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

系统设计题

本节属于《面试准备》。

Core Idea

AI Agent

系统设计题通常要求你设计一个可上线的业务系统，例如企业知识助手、自动报告生成、客服 Agent 或研发助手。

答题结构建议使用：需求澄清、核心流程、架构分层、数据与工具、可靠性、安全、评估、成本与迭代计划。

Engineering Notes

在工程实践中，最重要的是把概念转化为可执行的边界：输入是什么，输出是什么，失败时如何处理，如何观测质量，如何回滚错误。

如果一个能力无法被测试、无法被记录、无法被复现，它就不应该直接进入生产级 Agent 的关键路径。

Common Pitfalls

- 把模型输出当成确定性事实。
- 用更长的 Prompt 掩盖系统边界不清。
- 在没有评估集的情况下反复调参。
- 忽略权限、成本、延迟和失败恢复。

Practical Checklist

- 明确本节概念在真实 Agent 系统中的位置。
- 区分模型能力、工程约束和业务规则。
- 为后续代码示例保留可测试、可评估的接口。

Next Step

本节是 v0.1 草稿，后续会补充代码示例、架构图、案例和练习。

Navigation

- [Back to Chapter Index](#)
- [Next](#)

